

VERİ YAPILARI

Python dilinde kullanılan veri yapılarının her biri nesnedir. Nesneler davranışları ve özellikleri olan varlıklardır. Dolayısıyla bu dildeki veri yapılarının her birinin kendine has **yöntemleri** (**method**) bulunur. C/C++/C# ve Java gibi dillerinde bulunan int, float gibi temel veri tiplerinin aksine Python dilinde tüm **değişkenler** (**variable**) aslında var olan bir **nesneye** (**object**) tanımlanan bir referanstır.

Python dilinde dahili veri tipleri olan **bool**, **int**, **float**, **complex** ve **str** gibi veri yapılarında **değişken tanımlama** (**definition**) sırasında nesne **yapıcıları** (**constructor**) örtük olarak kullanılır. Aslında **list**, **tuple**, **set** ve **dict** veri yapılarında ise sırasıyla **[]**, **()**, **{}**, **{}** karakterleri ile örtük olarak bu veri yapılarının yapıcıları çağrılır.

Dizi (array)

Şu ana kadar en basit **veri yapıları** (**data structure**) olan **değişkenlerle** (**variable**) karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (**array size**) denir. Elemanların sırası **indis** (**index**) olarak adlandırılır. Kümenin aksine aynı elemanlar dizide farklı konumlarda birkaç kez bulunabilir. Elemanlara terim adı da verilir.

Örneğin Elemanları 1'den 10'a kadar sayıların karesinden oluşan bir dizi; $(a_k)_{k=1}^{10}, a_k = k^2$ veya $(a_1, a_2, a_3, \dots, a_{10})$ veya $(1, 4, 9, \dots, 100)$ olarak gösterilir. Örneğin;

- (K, İ, T, A, P) Dizisi ile (P, A, T, İ, K) dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları farklıdır.
- (1, 3, 5, 1, 2, 1, 7) Dizisinde birden farklı indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.
- Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler yani **uzunluğu** (**size**) belirli diziler kullanılır.

Python dilindeki **diziler** (**array**), C, C++ ve C# ve Java gibi geleneksel programlama dillerindeki dizilere benzer. Aynı veri tipindeki üyelerden oluşan homojen bir veri yapısıdır. **Listeler** (**list**), dizilere benzer ancak elemanları farklı veri tiplerinden oluşabilir. Yani bir **liste** (**list**), homojen ya da heterojen elemanların bir koleksiyonu olabilir.

Bireysel elemanlara **indisler** (**index**) aracılığıyla erişilebilir. Python dizilerinde indisler her zaman sıfırdan başlar. Bir dizi aşağıdaki şekilde tanımlanır;

```
diziKimliği = array(tipKodu, [Başlatıcılar])
```

Python dilinde dizileri kullanmak için standart **array** modülünü **içe aktarmanız** (**import**) gerekir.

```
from array import *
tamsayıDizisi = array('i', [1, 2, 3, 1, 3])
print(tamsayıDizisi) #diziyi konsola yazma
print(tamsayıDizisi[0]) #ilk elemanı konsola yazma
print(tamsayıDizisi[1]) #ikinci elemanı konsola yazma
#Tüm elemanları yineleyici ile konsola basma:
for i in tamsayıDizisi:
    print(i)
diziuzunluğu=len(tamsayıDizisi)
print(diziuzunluğu) #5
```

Diziyi oluşturan ilk parametre elemanların veri tipini belirler. Bu parametre;

Parametre	Eleman Veri Tipi
b	1 bayt boyutunda işaretli tam sayı
B	1 bayt boyutunda işaretli tam sayı
u	2 bayt boyutunda Unicode karakter

h	2 bayt boyutunda işaretli tam sayı
H	2 bayt boyutunda işaretli tam sayı
i	2 bayt boyutunda işaretli tam sayı
I	2 bayt boyutunda işaretli tam sayı
l	4 bayt boyutunda işaretli tam sayı
L	4 bayt boyutunda işaretli tam sayı
f	4 bayt boyutunda kayan noktalı sayı
d	8 bayt boyutunda kayan noktalı sayı

Tablo 16. Dizi Eleman Veri Tiplerini Belirleyen Parametreler

Dizilerin sonuna bir ekleme `append` ile veya indis belirterek istenilen yere `insert` ile yeni eleman eklenebilir.

```
from array import *
tamsayıDizisi = array('i', [1,2,3,1,3])
tamsayıDizisi.append(100) # dizi sonuna 100 sayısını ekleme
print(tamsayıDizisi) # array('i', [1, 2, 3, 1, 3, 100])
tamsayıDizisi.insert(0,200) # ilk eleman olarak 200 ekleme
print(tamsayıDizisi) # array('i', [200, 1, 2, 3, 1, 3, 100])
```

Dizilerin sonundaki eleman `pop` ile silinir veya indis belirterek istenilen konumdaki eleman `remove` ile yeni eleman silinebilir.

```
from array import *
tamsayıDizisi = array('i', [1,2,3,1,3])
tamsayıDizisi.pop() # dizi sonundaki eleman silinir
print(tamsayıDizisi) # array('i', [1, 2, 3, 1])
tamsayıDizisi.remove(2) # 2 değerine sahip eleman silinir
print(tamsayıDizisi) # array('i', [1, 3, 1])
```

Bir dizi aynı tipten elemanları olan bir başka dizi ile `extend` yöntemi ile birleştirilebilir. Ayrıca bir listeden de dizi sonuna `fromList` yöntemi ile eleman eklenebilir.

```
from array import *
dizi = array('i', [1,2,3,4,5])
eklenecekDizi = array('i', [7,8,9,10])
dizi.extend(eklenecekDizi)
print(dizi) #array('i', [1, 2, 3, 4, 5, 7, 8, 9, 10])
liste=[11,12,13]
dizi.fromList(liste)
print(dizi) #array('i', [1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13])
```

Argüman olarak verilen değer ile eşleşen ilk indisi `index` döndürür.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.index(4)) #3
print(dizi.index(1)) #0
```

Dizideki eleman konumları tersine çevrilmek istenirse `reverse` yöntemi kullanılır.

```
from array import *
dizi = array('i', [1,2,3,4,1])
dizi.reverse()
print(dizi) #array('i', [1, 4, 3, 2, 1])
```

Bellekteki dizi tamponunun (`buffer`) başlangıç adresini ve dizideki öge sayısını `buffer_info` yöntemi verir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.buffer_info()) #(140514698296784, 5)
```

Bir öğenin bir dizide kaç kez görüldüğünü `count` yöntemi döndürecektir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
print(dizi.count(1)) #2
```

Bir `dizgi(string)` `encode` yöntemiyle her bir karakteri sayıya kodlanır. Ardından bayt dizisine `frombytes` yöntemi ile eklenebilir. Tersine bir bayt dizisinden `decode` ile karakterlere dönüştürülecek sayılara dönüştürülebilir.

```
from array import *
baytdizizi = array('b')
baytdizizi.frombytes('İlhan'.encode())
print(baytdizizi) #array('b', [-60, -80, 108, 104, 97, 110])
metin=baytdizizi.tobytes()
print(metin) #b'\xc4\xbc\x01han'
print(metin.decode()) #İlhan
```

Dizilerde bir kısım hazır/dahili (bult-in) fonksiyonlar kullanılır;

```
from array import *
dizi = array('i', [1,2,3,4,1])
print('Eleman Sayısı:', len(dizi)) #5
print('En Küçük Eleman:', min(dizi)) #1
print('En Büyük Eleman:', max(dizi)) #4
```

Bu duruma örnek `len` fonksiyonunu içeren olarak 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani `sapma (deviation)` ve karesi alınmış sapmalar yani `varyans (variance)` ile standart sapmayı yazdıran program örnek verilebilir.

```
from math import sqrt
satışlar=[100.0,850.0,500.0,600.0,750.0,650.0,450.0,800.0,900.0,110.0]
toplam=0.0
for satış in satışlar:
    toplam+=satış
ortalama=toplam/len(satışlar)
print('Ortalama', ortalama)
varyanstoplam=0.0
for satış in satışlar:
    sapma=satış-ortalama
    print('Sapma',sapma)
    varyanstoplam=sapma*sapma
standartsapma=sqrt(varyanstoplam/len(satışlar))
print('Standart Sapma',standartsapma)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Ortalama 571.0
Sapma -471.0
Sapma 279.0
Sapma -71.0
Sapma 29.0
Sapma 179.0
Sapma 79.0
Sapma -121.0
Sapma 229.0
Sapma 329.0
Sapma -461.0
Standart Sapma 145.7810001337623
```

Bir dizide indis negatif kullanılabilir. Bu durumda son elemandan geriye doğru elemana erişilir

```
from array import *
dizi = array('i', [1,2,3,4,1])
```

```
print(dizi[-2]) #4 #sondan ikinci eleman
print(dizi[-4]) #2 #sondan dördüncü eleman
```

Dizilerde **dilimleme (slicing)**, belirli bir diziden alt dizi oluşturmanın bir yoludur. Bu işlemde, dizinin bir kısmını veya parçasını çıkarırız. Python'da dilimleme işlemini gerçekleştirmek için dilim işlecini `[:]` kullanırız.

```
from array import *
dizi = array('i', [1,2,3,4,1])
altdizi = dizi[0:2] #ilk elemandan itibaren 2. indise kadar elemanı alarak alt dizi oluştur
print(altdizi) #array('i', [1, 2])
altdizi2 =dizi[1:3] #ikinci elemandan itibaren 3. indise kadar elemanı alarak alt dizi oluştur
print(altdizi2) #array('i', [2, 3])
```

Dilimlemede negatif indisler de kullanılabilir.

```
from array import *
dizi = array('i', [1,2,3,4,1])
altdizi = dizi[-3:-1] # sondan 3. elemandan sondan 1. elamana kadar alt dizi oluştur
print(altdizi) # array('i', [3, 4])
altdizi2 =dizi[-4:-2] # sondan 4. elemandan sondan 2. elamana kadar alt dizi oluştur
print(altdizi2) # array('i', [2, 3])
```

Liste (list)

Listeler (list) dizilerden farklı olarak homojen veri içermezler. Yani aşağıdaki gibi elemanları her tipten nesne olabilir ve aynı yöntemlere sahiptir.

```
liste=[1,'a',3.14, 1+2j, [4,5,6],(7,8,9)]
print(liste) #[1, 'a', 3.14, (1+2j), [4, 5, 6], (7, 8, 9)]
```

C/C++, C# ve Java dillerinin aksine iki boyutlu diziler yani **matrisler (matrix)**, liste içeren listeler şeklinde tanımlanabilir.

```
matris=[[1,2,3],[4,5,6],[7,8,9]]
print(matris[0]) #[1, 2, 3] # birinci satır
print(matris[1]) #[4, 5, 6] # ikinci satır
print(matris[2]) #[7, 8, 9] # üçüncü satır
print(matris[0][1]) #2 # birinci satır ikinci sütun elemanı
print(matris[1][1]) #5 # ikinci satır ikinci sütun elemanı
print(matris[2][1]) #8 # üçüncü satır ikinci sütun elemanı
matris[2][2]=100
print(matris) #[[1, 2, 3], [4, 5, 6], [7, 8, 100]]
```

Benzer şekilde üç boyutlu ya da çok boyutlu diziler tanımlanabilir.

```
üçboyutlu=[[11,12,13],[14,15,16],[17,18,19]],\
           [[21,22,23],[24,25,26],[27,28,29]],\
           [[31,32,33],[34,35,36],[37,38,39]]]
print(üçboyutlu[0]) # [[11, 12, 13], [14, 15, 16], [17, 18, 19]] # ilk matris
print(üçboyutlu[1]) # [[21, 22, 23], [24, 25, 26], [27, 28, 29]] # ikinci matris
print(üçboyutlu[2][0]) # [31, 32, 33] # üçüncü matrisin ilk satırı
print(üçboyutlu[1][0][1]) # 22 # ikinci matrisin ilk satırının ikinci sütun elemanı
```

Dizilerde olan ve dizi ve liste nesnelerinin kendine has yöntemleri vardır;

Yöntem	Açıklama
append()	Listenin sonuna bir eleman ekler
clear()	Tüm elemanları siler
copy()	Listenin bir kopyasını döndürür
count()	Belirtilen değerde listenin sahip olduğu eleman sayısını döndürür
extend()	Listeye bir başka liste ekleyerek genişletir
index()	Verilen bir değerın listede bulunan ilk indisini döner

insert()	Belirtilen indis konumuma yeni bir eleman ekler
pop()	Belirtilen indis konumundan bir elemanı siler
remove()	Belirtilen değerdeki ilk elemanı siler
reverse()	Listeyi ters çevirir
sort()	Değerlere göre listeyi sıralar

Tablo 17. Dizi ve Liste Nesnesi Yöntemleri

Liste yöntemleri, bir listenin son giren elemanı ilk çıkar (**Last In First Out-LIFO**) mantığı yani **yığın (stack)** mantığında kullanmayı da sağlar;

```
yığın=[1,2,3]
yığın.append(4) # sonuna 4 eklendi
yığın.append(5) # sonuna 5 eklendi
çıkan=yığın.pop()
print(çıkan) #5
```

Bir listedeki yinelenen değerleri kaldırmak, listeyi bir kümeye (yani, farklı nesnelerin sırasız bir koleksiyonu) dönüştürerek yapılabilir.

```
meyveler = ["armut", "elma", "erik", "elma", "armut"]
tekilmeyveler=list(set(meyveler))
print(tekilmeyveler) #['erik', 'armut', 'elma']
```

Liste ve dizilerde elemanlar üzerinde **yinelemeler (iteration)** aşağıdaki şekilde yapılabilir;

```
liste=[10,20,30,40,50]
for (indis,eleman) in enumerate(liste):
    print('{}.indisteki eleman={}'.format(indis,eleman))

for i in range(0,len(liste)):
    print(liste[i])
```

Bu kod çalıştığında vereceği çıktı aşağıda verilmiştir;

```
0.indisteki eleman=10
1.indisteki eleman=20
2.indisteki eleman=30
3.indisteki eleman=40
4.indisteki eleman=50
10
20
30
40
50
```

Listede bir elemanın varlığı **in** **işleci (operator)** ile test edildiği daha önce anlatılmıştı. Bir **yineleme (iterable)** içerisinde tüm değerlerin **True** olarak değerlendirip değerlendirmedini belirlemek için dahili **all()** fonksiyonunu kullanabilirsiniz.

```
liste=[1,2,3,0] #en az bir elemanı 0
all(liste) #False
liste=['a','b',0] #en az bir elemanı 0
all(liste) #False
liste=['a','b','c'] #hiç 0 değerli elemanı yok
all(liste) #True
```

Benzer şekilde, dahili **any()** fonksiyonu bir yinelemede bir veya daha fazla değer **True** olarak değerlendirip değerlendirmedini belirler.

```
liste=['a','b','c']
any(liste) #True
liste=[0,0,0] #Tüm değerler 0
any(liste) #False
```

Tersinen bir liste döndüren ters yani `reversed()` dahili fonksiyonunu kullanarak tersine bir yineleyici elde ederiz.

```
sayılar=[1,2,3,4]
for eleman in reversed(sayılar):
    print(eleman, end='-') #4-3-2-1-
terstensayılar=sayılar[::-1] #yineleyici değil de tersine bir dizi için::-1 kullanılır
print(terstensayılar) # [4, 3, 2, 1]
```

Bir **liste kavrama** (**list comprehension**), bir **ifade** (**expression**) ile liste üzerinde **yineleme** (**iterable**) ile her bir elemana uygulayarak yeni bir liste oluşturur. Sözde kodu aşağıda verilmiştir;

```
[ <expression> for <element> in <iterable> ]
#yada
[ <expression> for <element> in <iterable> if <condition> ]
```

Aşağıda bir listedeki sayıların karelerinden yeni bir liste oluşturan örnek verilmiştir;

```
liste=[1,2,3,4,5]
kareler=[x*x for x in liste]
print(kareler) #[1, 4, 9, 16, 25]

kelimeler=['ilhan','Özkan', 'trabzon']
büyükkelimeler=[s.upper() for s in kelimeler]
print(büyükkelimeler) #['ILHAN', 'ÖZKAN', 'TRABZON']

onakadarçiftsayılar=[x for x in range(10) if x % 2 == 0]
print(onakadarçiftsayılar) # [0, 2, 4, 6, 8]

yıldızlı=[x if x in 'aeiou' else '*' for x in 'Şifre']
print(yıldızlı) # ['*', 'i', '*', '*', 'e']
```

İki liste **+** işleci ile birleştirilebilir.

```
liste1=[1,2,3]
liste2=['a','b','c']
birleşikliste=liste1+liste2
print(birleşikliste) #[1, 2, 3, 'a', 'b', 'c']
```

Dahili **sıkıştırma** (**zip**) fonksiyonu, iki listenin karşılıklı elemanlarından oluşan bir **demet** (**tuple**) üretir. Eleman sayısı az olan kadar demet üretilir.

```
liste1=[1,2,3] #3 elemanlı
liste2=['a','b','c'] #üç elemanlı
liste3=[10.5, 20.6] #iki elemanlı
for x in zip(liste1,liste2):
    print(x)
for x in zip(liste1,liste3):
    print(x)
```

Programın çıktısı aşağıdaki gibi olacaktır;

```
(1, 'a')
(2, 'b')
(3, 'c')
(1, 10.5)
(2, 20.6)
```

Bir listenin belirli sayıda elemanı olması isteniyorsa aşağıdaki gibi tanımlanabilir.

```
liste1 = [None] * 10 #veri tipi belirlenmemiş 10 elemanlı liste
print(liste1) #[None, None, None, None, None, None, None, None, None, None]
liste2= ['a']*5 #veri tipi str belirlenmiş 5 elemanlı liste
print(liste2) #['a', 'a', 'a', 'a', 'a']
```

Dizgi (str)

Python dilinde metinleri işlemek için **dizgi** (**string**) veri yapısı **str** ile tanımlanır. Dizgiler, Unicode karakterlerinin sıralı bir **dizisidir** (**array**). C++ dilindeki **string** veri yapısı gibidir. Yani tüm dizgi nesnelere ilişkin yöntemler, orijinal dizgiyi bozmaz ve yeni bir **str** nesnesi döndürür.

Yöntem	Açıklama
capitalize()	İlk harfi büyüterek döndürür
casefold()	Tüm harfleri küçüğe çevirerek döndürür
center()	Metni belirtilen karakter sayısına göre ortasına yerleştirir
count()	Belirtilen metnin dizgi içerisinde kaç defa geçtiğini döndürür
encode()	Metnin kodlanmış (encoded) halini döndürür.
endswith()	Belirtilen metin ile bitiyorsa dizgiyi döndürür.
expandtabs()	Belirtilen sekme miktarına göre dizgi içerisindeki sekmeleri (\t) genişleterek döndürür.
find()	Belirtilen metni dizgi içerisinde arar.
format()	Dizgi metnini biçimlendirme metni olarak kullanır. Biçimlendirilmiş metni döndürür.
format_map()	Dizgiyi bir sözlükten alınan değerler ile biçimlendirir
index()	Dizgide belirtilen bir değeri arar ve bulunduğu konumu döndürür
isalnum()	Dizgideki tüm karakterler alfasayısal (alphanumeric) ise True değerini döndürür
isalpha()	Dizgideki tüm karakterler alfabetik ise True değerini döndürür
isascii()	Dizgideki tüm karakterler ASCII karakterleriyse True değerini döndürür
isdecimal()	Dizgideki tüm karakterler ondalık sayıysa True değerini döndürür
isdigit()	Dizgideki tüm karakterler rakamsa True değerini döndürür
isidentifier()	Dizgi bir değişken kimliği (identifier) ise True değerini döndürür
islower()	Dizgideki tüm karakterler küçük harf ise True değerini döndürür
isnumeric()	Dizgideki tüm karakterler sayısal ise True değerini döndürür
isprintable()	Dizgideki tüm karakterler yazıcıda yazdırılabilirse True değerini döndürür
isspace()	Dizgideki tüm karakterler beyaz boşluksa (whitespace) True değerini döndürür
istitle()	Dizgi bir başlığın kurallarına (Her Sözcüğün İlk Harfi Büyük) uyuyorsa True değerini döndürür
isupper()	Dizgideki tüm karakterler büyük harf ise True değerini döndürür
join()	Bir yineleyicinin (iterable) öğelerini bir dizgiye dönüştürür
ljust()	Dizginin sola yaslanmış halini döndürür
lower()	Bir dizgiyi küçük harfe dönüştürür
lstrip()	Dizginin soldan kırılmış (boşluğu alınmış) halini döndürür
maketrans()	Çevirilerde kullanılacak bir çeviri tablosu döndürür
partition()	Dizenin üç parçaya bölündüğü bir demet (tuple) döndürür
replace()	Belirtilen bir değer bir başka değerle değiştirildiği bir dizgi döndürür
rfind()	Dizgide belirtilen bir değer aranır ve bulunduğu son konumu döndürür
rindex()	Dizgide belirtilen bir değer aranır ve bulunduğu son konumu döndürür
rjust()	Dizginin sağa yaslanmış bir sürümünü döndürür
rpartition()	Dizginin üç parçaya bölündüğü bir demet (tuple) döndürür
rsplit()	Dizgiyi sondan başa doğru belirtilen ayırıcı ile böler ve bir liste döndürür
rstrip()	Dizginin sağdan kırılmış (boşlukları alınmış) şeklini döndürür
split()	Dizgiyi belirtilen ayırıcı ile böler ve bir liste döndürür
splitlines()	Dizgiyi satır sonlarında böler ve bir liste döndürür

Yöntem	Açıklama
startswith()	Dizgi belirtilen değerle başlıyorsa True değerini döndürür
strip()	Dizinin kırılmış (boşlukları alınmış) şeklini döndürür
swapcase()	Harfleri değiştirir, küçük harf büyük harfe, büyük harf küçük harfe dönüşür ve dönüşmüş halini döndürür.
title()	Her kelimenin ilk karakterini büyük harfe dönüşmüş halini döndürür
translate()	Dizginin çevrilmiş halini döndürür
upper()	Bir dizgiyi büyük harfe dönüştürür
zfill()	Dizginin başını belirtilen sayıda 0 değerle doldurur

Tablo 18. Dizgi Nesnesi Yöntemleri

Aşağıdaki örnekte dizgi nesnesine ilişkin yöntemlerin birkaçı yer almaktadır.

```
metin=' ilhan özkan '
print(metin.lower()) # ilhan özkan
print(metin.upper()) # ILHAN ÖZKAN
print(metin.title()) # İlhan Özkan
print(metin.strip()) #ilhan özkan
print(metin.find('kan')) #9
print(len(metin)) #13 #Dahili fonksiyonlar da dizgilerle kullanılabilir.
print(metin.islower()) #True
print(metin.isprintable()) #True
```

Dizgiler üzerinde toplama (+) işleci sağ ve solundaki metni birleştirir. Çarpma (*) işleci ise **işlenenlerden (operand)** biri tamsayı olmak kaydıyla metni belirtilen tamsayı kadar çoğaltır;

```
metin1='İlhan'
metin2='özkan'
print(metin1,metin2) #İlhan özkan
print(metin1+metin2) #İlhanözkan
print(metin1*2) #İlhanİlhan
print(metin1*4) #İlhanİlhanİlhanİlhan
print(2*metin1) #İlhanİlhan
```

Birleştirilecek dizgi **değişmezleri (literal)** yan yana yazılarak da dizgiler oluşturulabilir;

```
metin='İlhan' " " "özkan" ',' ""Merhaba!""
print(metin) #İlhan özkan,Merhaba!
```

Dizgilere tek veya çift tırnak arasında yazılan değişmezler ile başlatılır. Metnin içerisinde geçecek tek ve çift tırnak karakterleri için değişmez içinde C dilindeki gibi ters bölü karakteri ile belirtilir.

```
metin='İlhan\'ın soyadı "Özkan" dır'
print(metin) #İlhan\'ın soyadı "Özkan" dır
metin="İlhan\'ın soyadı \"Özkan\" dır"
print(metin) #İlhan\'ın soyadı "Özkan" dır
```

Ters bölü ile belirtilen bu karakterlere **kaçış dizisi (escape sequence)** karakterleri adı verilir.

Karakter	Açıklama
\'	Tek Tırnak
\"	Çift Tırnak
\\	Ters Bölü
\n	Yeni satır (new line): İmleç (cursor), ↵ tuşu davranışı sergiler.)
\b	İmleç, bir karakter geri gider. Yani geri (backspace) tuşu ⇐ davranışı sergiler.
\r	Satır başı (carriage return): İmleç, ⇐ tuşu davranışı sergiler.)
\t	Sekme (Konsolda 8 ve katları sütuna ilerlenir): İmleç, → tuşu davranışı sergiler.
\f	Yeni sayfa (form feed): Yazıcıda yeni sayfaya geç anlamında kullanılır.
\ooo	Sekizlik (octal) Sayı Sisteminde Karakter Kodu
\xhh	Onlatılık (hexadecimal) Sayı Sisteminde Karakter Kodu

Tablo 19. Kaçış Dizisi Karakterleri

Aşağıda birkaç örnek verilmiştir;

```
metin = "\110\145\154\154\157"
print(metin)
metin = "\x48\x65\x6c\x6c\x66"
print(metin)
metin="İlhan\nÖzkan" # \n ile konsolda satır başı yapılır
print(metin)
metin="İlhan\tÖzkan" #\t ile konsolda 8 ve katları olan sütuna ilerlenir.
print(metin)
metin="İlhan\bÖzkan" #\b ile konsolda son konumdan bir karakter geri ilerlenir.
print(metin)
metin="Merhaba\rİlhan" #\r ile konsolda mevcut satırın başına ilerlenir.
print(metin)
```

Örneğin çalıştırıldığında çıktısı aşağıdaki gibi olacaktır

```
Hello
Hello
İlhan
Özkan
İlhan   Özkan
İlhaÖzkan
İlhanba
```

Demet (tuple)

Demet (tuple), değiştirilemez bir değerler listesidir ve aşağıdaki şekilde oluşturulabilirler.

```
demet1=() #elemanı olmayan bir demet (tuple)
demet2='a','b','c'
demet3='a',
demet4=('a')
demet5=('a',)
demet6=tuple(range(5)) #yapıcı kullanarak aralık nesnesinden demet yapma
print(demet6) #(0, 1, 2, 3, 4)
demet7=tuple("Demet") #yapıcı kullanarak str nesnesinden demet yapma
print(demet7) #('D', 'e', 'm', 'e', 't')
```

Demeti oluşturan nesneler **değiştirilemez** (immutable). Ancak yeni elemanlar ile genişletilebilir veya iki demet birleştirilebilir.

```
demet=(1,2,3,4)
#demet[2]=100 #TypeError: 'tuple' object does not support item assignment
baskaDemet=('a','b','c')
yeniDemet=demet+baskaDemet
print(yeniDemet) #(1, 2, 3, 4, 'a', 'b', 'c')
yeniDemet+=(1+4j,"İlhan")
print(yeniDemet) #(1, 2, 3, 4, 'a', 'b', 'c', (1+4j), 'İlhan')
```

Demetlere ilişkin yöntemler aşağıda verilmiştir.

Yöntem	Açıklama
count()	Belirtilen değerin demet içinde kaç kez bulunduğunu döndürür.
index()	Belirtilen değeri demet içinde arar ve bulunduğu yeri döndürür.

Bir demetin yalnızca bazı öğelerine ihtiyaç duyuluyorsa, altçizgi sembolü (_) geçici bir değişken adı olarak kullanılabilir ve **yer tutucu** (placeholder) olarak görev yapar.

```
demet = 1, 2, 3, 4
_, x, y, _ = demet # ikinci eleman x , üçüncü eleman y
if x == 2:
```

```
print(x+y) #5
```

Benzer şekilde bir demetin bitişim birkaç öğesine ihtiyaç duyuluyorsa, yıldız sembolü (*) ile geçici bir değişken adı kullanılabilir ve buna **paket açma** (**unpacking**) adı verilir.

```
demet = (1, 2, 3, 4)
a,*b,c = demet # ikinci ve üçüncü elemanı içeren bir liste b değişkenini oluşturur
print(b) # [2,3]
```

Küme (set)

Küme (set), herhangi bir veri tipinde ve **tekil (unique)** elemanlardan oluşan sırasız bir koleksiyondur. Bir nesne, ömrü boyunca asla değişmeyen bir **karma (hash)** değerine sahipse (ki bu nesne bir **hash()** yöntemine ihtiyaç duyar) ve diğer nesnelerle **karşılaştırılabilir** (bir **eq()** yöntemine ihtiyaç duyar) **karmalanabilir** (**hashable**). Eşitliği karşılaştıran karmalanabilir nesnelerin, aynı karma değerine sahip olması gerekir. Karma, bir nesneye eşsiz bir sayı olarak karşılık gelen değerlerdir.

```
a = {1, 2, 'a'}
print(a)
# Çıktı: {1, 2, 'a'}
b = {1, 1, 2, 2} # bir değer bir kümede yalnızca 1 kez yer alabilir
print(b)
# Çıktı: {1, 2}
c = set() # boş küme oluşturma
print(c)
# Çıktı: set()
d = set([1, 2, 3, 3, 4]) # listeden küme oluşturma
print(d)
# Çıktı: {1, 2, 3, 4}
```

Kümeler sırasız olduğundan indeks kullanılarak erişilemez. Ancak for döngüsü ile elemanlar üzerinde gezinilebilir.

```
renkler = {'kırmızı', 'yeşil', 'mavi'}
for renk in renkler:
    print(renk)
# Çıktı: mavi
# Çıktı: yeşil
# Çıktı: kırmızı
# in işleci ile eleman kontrolü
'kırmızı' in renkler
# Çıktı: True
'sarı' in renkler
# Çıktı: False
```

Küme elemanlarına aşağıdaki şekilde ekleme ve çıkarma yapılabilir;

```
meyveler = {'elma', 'armut'}
print(meyveler)
# Çıktı: {'elma', 'armut'}

# add() - tek eleman ekleme
meyveler.add('muz')
print(meyveler)
# Çıktı: {'elma', 'armut', 'muz'}

# update() - birden fazla eleman ekleme
meyveler.update(['portakal', 'çilek'])
print(meyveler)
# Çıktı: {'elma', 'armut', 'muz', 'portakal', 'çilek'}

# remove() - eleman çıkarma (yoksa hata verir)
meyveler.remove('armut')
```

```
print(meyveler)
# Çıktı: {'elma', 'muz', 'portakal', 'çilek'}

# discard() - eleman çıkarma (yoksa hata vermez)
meyveler.discard('kayısı') # hata vermez
meyveler.discard('muz')
print(meyveler)
# Çıktı: {'elma', 'portakal', 'çilek'}

# pop() - rastgele bir eleman çıkarır
eleman = meyveler.pop()
print(f"Çıkarılan: {eleman}, Kalan: {meyveler}")
# Çıktı: Çıkarılan: elma, Kalan: {'portakal', 'çilek'}

# clear() - tüm elemanları siler
meyveler.clear()
print(meyveler)
# Çıktı: set()
```

Kümeler matematikten bildiğimiz küme işlemlerini destekler. Aşağıda örnekleri verilmiştir;

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# Birleşim (union) - |
print(A | B)
# Çıktı: {1, 2, 3, 4, 5, 6, 7, 8}
print(A.union(B))
# Çıktı: {1, 2, 3, 4, 5, 6, 7, 8}

# Kesişim (intersection) - &
print(A & B)
# Çıktı: {4, 5}
print(A.intersection(B))
# Çıktı: {4, 5}

# Fark (difference) - -
print(A - B) # A'da olup B'de olmayanlar
# Çıktı: {1, 2, 3}
print(A.difference(B))
# Çıktı: {1, 2, 3}
print(B - A) # B'de olup A'da olmayanlar
# Çıktı: {6, 7, 8}

# Simetrik fark (symmetric_difference) - ^
print(A ^ B) # Ya A'da ya B'de ama ikisinde birden olmayanlar
# Çıktı: {1, 2, 3, 6, 7, 8}
print(A.symmetric_difference(B))
# Çıktı: {1, 2, 3, 6, 7, 8}
```

Alt küme işlemleri da aşağıdaki şekilde yapılır;

```
A = {1, 2, 3}
B = {1, 2, 3, 4, 5}
C = {1, 2}

# issubset() - alt küme mi?
print(A.issubset(B)) # A, B'nin alt kümesi mi?
# Çıktı: True
print(C.issubset(A)) # C, A'nın alt kümesi mi?
# Çıktı: True

# issuperset() - üst küme mi?
```

```
print(B.issuperset(A)) # B, A'nın üst kümesi mi?
# Çıktı: True
print(A.issuperset(C)) # A, C'nin üst kümesi mi?
# Çıktı: True

# isdisjoint() - kesişimleri boş mu?
D = {6, 7, 8}
print(A.isdisjoint(D)) # A ve D'nin ortak elemanı var mı?
# Çıktı: True
print(A.isdisjoint(B))
# Çıktı: False
```

Küme eleman sayısı ve küme kopyalama;

```
sayilar = {1, 2, 3, 4, 5}

# len() - eleman sayısı
print(len(sayilar))
# Çıktı: 5

# copy() - kümeyi kopyala
yeni_sayilar = sayilar.copy()
yeni_sayilar.add(6)
print(sayilar)
# Çıktı: {1, 2, 3, 4, 5}
print(yeni_sayilar)
# Çıktı: {1, 2, 3, 4, 5, 6}
```

Değiştirilemez kümeler (**frozenset**) de tanımlanabilir. Değiştirilemez kümeler, küme gibidir ancak elemanları **değiştirilemez** (**immutable**). Bu sayede sözlüklerde anahtar veya başka bir kümenin elemanı olarak kullanılabilir.

```
donuk_kume = frozenset([1, 2, 3, 4])
print(donuk_kume)
# Çıktı: frozenset({1, 2, 3, 4})

# Değiştirilemez
# donuk_kume.add(5) # AttributeError hatası verir

# Küme işlemleri çalışır
diger_kume = frozenset([3, 4, 5, 6])
print(donuk_kume | diger_kume)
# Çıktı: frozenset({1, 2, 3, 4, 5, 6})

# Sözlükte anahtar olarak kullanılabilir
sozluk = {donuk_kume: "değer"}
print(sozluk)
# Çıktı: {frozenset({1, 2, 3, 4}): 'değer'}
```

Kümeye tekrarlanan elemanları kaldırma aşağıdaki şekilde yapılabilir;

```
liste = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
tekil_liste = list(set(liste))
print(tekil_liste)
# Çıktı: [1, 2, 3, 4]
```

İki farklı kümenin ortak elemanlarını bulma aşağıdaki şekilde yapılabilir;

```
ogrenci1_dersler = {'matematik', 'fizik', 'kimya', 'biyoloji'}
ogrenci2_dersler = {'fizik', 'kimya', 'tarih', 'edebiyat'}
ortak_dersler = ogrenci1_dersler & ogrenci2_dersler
print(ortak_dersler)
# Çıktı: {'fizik', 'kimya'}
```

İki farklı kümenin farklı elemanlarını bulma aşağıdaki şekilde yapılabilir;

```
tum_dersler = ogrenci1_dersler | ogrenci2_dersler
print(tum_dersler)
# Çıktı: {'matematik', 'fizik', 'kimya', 'biyoloji', 'tarih', 'edebiyat'}
sadece_ogrenci1 = ogrenci1_dersler - ogrenci2_dersler
print(sadece_ogrenci1)
# Çıktı: {'matematik', 'biyoloji'}
```

Sözlük (dict)

Sözlük (**dict**), **anahtar-değer** (**key-value**) **çiftlerini** (**pair**) tekil olarak tutan bir koleksiyondur. Anahtarlar **karmalanabilir** (**hashable**).

```
a = {1: 'Elma', 2: 'Armut', 3: 'Muz'}
print(a)
# Çıktı: {1: 'Elma', 2: 'Armut', 3: 'Muz'}

b = {'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}
print(b)
# Çıktı: {'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}

# Boş sözlük oluşturma
c = {}
print(c)
# Çıktı: {}
d = dict()
print(d)
# Çıktı: {}
```

Sözlük elemanlarına erişim aşağıdaki şekilde yapılabilir;

```
ogrenci = {'isim': 'Ali', 'yas': 20, 'bolum': 'Bilgisayar'}

# Köşeli parantez ile erişim
print(ogrenci['isim'])
# Çıktı: Ali
print(ogrenci['yas'])
# Çıktı: 20

# get() metodu ile erişim
print(ogrenci.get('bolum'))
# Çıktı: Bilgisayar
print(ogrenci.get('not')) # Olmayan anahtar, None döner
# Çıktı: None
print(ogrenci.get('not', 0)) # Varsayılan değer belirtme
# Çıktı: 0

# Köşeli parantez ile olmayan anahtar KeyError verir
# print(ogrenci['not']) # KeyError: 'not'
```

Sözlüğe eleman ekleme ve güncelleme aşağıdaki şekilde yapılabilir;

```
kitap = {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan'}
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan'}

# Yeni anahtar-değer ekleme
kitap['yil'] = 2025
kitap['sayfa'] = 500
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yil': 2025, 'sayfa': 500}
```

```
# Mevcut değeri güncelleme
kitap['sayfa'] = 550
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yıl': 2025, 'sayfa': 550}

# update() ile birden fazla ekleme/güncelleme
kitap.update({'dil': 'Türkçe', 'fiyat': 150})
print(kitap)
# Çıktı: {'baslik': 'Python Programlama', 'yazar': 'İlhan Özkan', 'yıl': 2025, 'sayfa': 550,
'dil': 'Türkçe', 'fiyat': 150}
```

Sözlükten eleman silme aşağıdaki şekilde yapılabilir;

```
urun = {'isim': 'Laptop', 'fiyat': 15000, 'stok': 5, 'kategori': 'Elektronik'}

# del ile silme
del urun['stok']
print(urun)
# Çıktı: {'isim': 'Laptop', 'fiyat': 15000, 'kategori': 'Elektronik'}

# pop() - anahtarı siler ve değeri döner
kategori = urun.pop('kategori')
print(f"Silinen: {kategori}, Kalan: {urun}")
# Çıktı: Silinen: Elektronik, Kalan: {'isim': 'Laptop', 'fiyat': 15000}

# popitem() - son eklenen anahtar-değer çiftini siler (Python 3.7+)
son = urun.popitem()
print(f"Silinen çift: {son}, Kalan: {urun}")
# Çıktı: Silinen çift: ('fiyat', 15000), Kalan: {'isim': 'Laptop'}

# clear() - tüm elemanları siler
urun.clear()
print(urun)
# Çıktı: {}
```

Sözlükte anahtar, değer ve anahtar-değer çiftleri üzerinde işlemler aşağıdaki şekilde yapılabilir;

```
notlar = {'Ali': 85, 'Ayşe': 92, 'Mehmet': 78, 'Fatma': 95}

# keys() - tüm anahtarları döner
print(notlar.keys())
# Çıktı: dict_keys(['Ali', 'Ayşe', 'Mehmet', 'Fatma'])

# values() - tüm değerleri döner
print(notlar.values())
# Çıktı: dict_values([85, 92, 78, 95])

# items() - tüm anahtar-değer çiftlerini döner
print(notlar.items())
# Çıktı: dict_items([('Ali', 85), ('Ayşe', 92), ('Mehmet', 78), ('Fatma', 95)])

# Döngü ile gezinme
for isim in notlar.keys():
    print(isim)
# Çıktı: Ali
# Çıktı: Ayşe
# Çıktı: Mehmet
# Çıktı: Fatma

for not_degeri in notlar.values():
    print(not_degeri)
# Çıktı: 85
# Çıktı: 92
```

```
# Çıktı: 78
# Çıktı: 95

for isim, not_degeri in notlar.items():
    print(f"{isim}: {not_degeri}")
# Çıktı: Ali: 85
# Çıktı: Ayşe: 92
# Çıktı: Mehmet: 78
# Çıktı: Fatma: 95

araba = {'marka': 'Toyota', 'model': 'Corolla', 'yil': 2023}

# in operatörü - anahtar var mı?
print('marka' in araba)
# Çıktı: True
print('renk' in araba)
# Çıktı: False

# not in operatörü
print('fiyat' not in araba)
# Çıktı: True

# len() - eleman sayısı
print(len(araba))
# Çıktı: 3
```

İç içe sözlükler aşağıdaki gibi işlenir;

```
okullar = {
    'lise1': {
        'isim': 'Atatürk Lisesi',
        'ogrenci_sayisi': 500,
        'sehir': 'Ankara'
    },
    'lise2': {
        'isim': 'Fen Lisesi',
        'ogrenci_sayisi': 300,
        'sehir': 'İstanbul'
    }
}

# İç içe sözlüğe erişim
print(okullar['lise1']['isim'])
# Çıktı: Atatürk Lisesi
print(okullar['lise2']['ogrenci_sayisi'])
# Çıktı: 300

# Tüm okullarda gezinme
for okul_id, okul_bilgi in okullar.items():
    print(f"\n{okul_id}:")
    for anahtar, deger in okul_bilgi.items():
        print(f"  {anahtar}: {deger}")

# lise1:
#   isim: Atatürk Lisesi
#   ogrenci_sayisi: 500
#   sehir: Ankara

# lise2:
#   isim: Fen Lisesi
#   ogrenci_sayisi: 300
#   sehir: İstanbul
```

Sözlüğe varsayılan değer `setdefault()` yöntemi ile eklenir;

```
sayac = {}

# setdefault() - anahtar yoksa ekler ve değeri döner
print(sayac.setdefault('a', 0))
# Çıktı: 0
print(sayac)
# Çıktı: {'a': 0}

# Anahtar varsa mevcut değeri döner
sayac['a'] = 5
print(sayac.setdefault('a', 0))
# Çıktı: 5
print(sayac)
# Çıktı: {'a': 5}

# Kelime sayma örneği
metin = "merhaba dünya merhaba python python python"
kelime_sayaci = {}
for kelime in metin.split():
    kelime_sayaci[kelime] = kelime_sayaci.get(kelime, 0) + 1
print(kelime_sayaci)
# Çıktı: {'merhaba': 2, 'dünya': 1, 'python': 3}
```

Sözlüğü kopyalama aşağıdaki şekilde yapılabilir;

```
orijinal = {'a': 1, 'b': 2, 'c': 3}

# Sığ kopyalama (shallow copy)
kopya1 = orijinal.copy()
kopya1['a'] = 100
print(orijinal)
# Çıktı: {'a': 1, 'b': 2, 'c': 3}
print(kopya1)
# Çıktı: {'a': 100, 'b': 2, 'c': 3}

# dict() ile kopyalama
kopya2 = dict(orijinal)
print(kopya2)
# Çıktı: {'a': 1, 'b': 2, 'c': 3}
```

Listelerde olduğu gibi sözlüklerde de **kavrama** (**comprehension**) vardır. Kavrama ile, bir **ifadenin** (**expression**) sözlük üzerinde **yineleme** (**iterable**) ile her bir elemana uygulayarak yeni bir sözlük oluşturulur.

```
# Basit sözlük comprehension
kareler = {x: x**2 for x in range(1, 6)}
print(kareler)
# Çıktı: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Koşullu sözlük comprehension
cift_kareler = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(cift_kareler)
# Çıktı: {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}

# İki listeden sözlük oluşturma
isimler = ['Ali', 'Ayşe', 'Mehmet']
yaslar = [25, 30, 28]
isim_yas = {isim: yas for isim, yas in zip(isimler, yaslar)}
print(isim_yas)
# Çıktı: {'Ali': 25, 'Ayşe': 30, 'Mehmet': 28}
```

```
# Sözlüğü ters çevirme
orijinal = {'a': 1, 'b': 2, 'c': 3}
ters = {deger: anahtar for anahtar, deger in orijinal.items()}
print(ters)
# Çıktı: {1: 'a', 2: 'b', 3: 'c'}
```

Aşağıda bir öğrenci sistemine ilişkin örnek verilmiştir;

```
ogrenci = {
    'numara': '12345',
    'ad': 'Ali',
    'soyad': 'Veli',
    'notlar': {
        'matematik': 85,
        'fizik': 78,
        'kimya': 92
    }
}

ortalama = sum(ogrenci['notlar'].values()) / len(ogrenci['notlar'])
print(f"{ogrenci['ad']} {ogrenci['soyad']}'nin ortalaması: {ortalama:.2f}")
# Çıktı: Ali Veli'nin ortalaması: 85.00
```

Sözlükle ilgili aşağıda bir başka telefon defleri örneği verilmiştir;

```
rehber = {
    'Ali': '555-1234',
    'Ayşe': '555-5678',
    'Mehmet': '555-9012'
}

# Arama
isim = 'Ayşe'
if isim in rehber:
    print(f"{isim}'nin telefonu: {rehber[isim]}")
else:
    print(f"{isim} rehberde yok")
# Çıktı: Ayşe'nin telefonu: 555-5678
```